

华东师范大学数据科学与工程学院实验报告

课程名称:当代人工智能	年级:22级	上机实践成绩:
指导教师:李翔	姓名:郭夏辉	学号:10211900416
上机实践名称:文本摘要	上机截止日期:2023年12月29日	上机实践编号:No.4

一、实验目的

文本摘要是指使用算法从一段较长文本中提取关键信息，生成简短、凝练的摘要。本次实验是文本摘要在医学领域的应用：从详细的病情描述中提取关键信息生成简短清晰的诊断报告。

医疗领域数据集（已脱敏）：

列名	数据类型	示例	示例
description（输入）病情描述	string	左侧顶骨局部骨质缺如；两侧侧脑室旁见点状密度减低。右侧额部颅板下见弧形脑脊液密度影。脑室系统扩大，脑沟、裂、池增宽。中线结构无移位。双侧乳突气化差，内见密度增高。	101 47 12 66 74 90 0 411 234 79 175
diagnosis（输出）诊断报告	string	左侧顶骨局部缺如，考虑术后改变；脑内散发缺血灶；右侧额部少量硬膜下积液；双侧乳突炎。	122 83 65 74 2 232 18 44 95

18000条训练数据，要自行划分验证集；2000条测试数据。

实验要求总结：

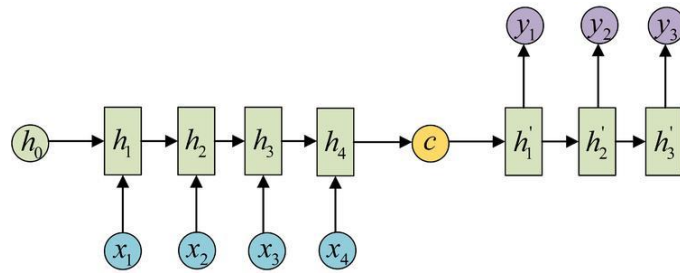
- 构建seq2seq模型完成本次文本摘要任务；
- Encoder/Decoder的选择包括但不限于RNN/LSTM/GRU/Transformer/BERT/ BART/T5/OPT等；选择一种Encoder-Decoder结构，使用哪种模型作为Encoder或Decoder不受限制，大家可以按照自己的硬件设备自行选择；在硬件条件允许的情况下，尽量使用多种不同的模型并进行详细的实验分析；
- 评估指标的选择：BLEU-4、ROUGE、CIDEr等；评估指标的使用不受限制，可以仅使用一种或使用多种更加全面的进行结果分析；

二、模型介绍

本次实验我选择了三种模型——BART-base、T5-small和GRU

我先来花一些篇幅来介绍一下BART和T5这两种都是基于Transformer架构的seq2seq模型，其中贯穿着就会介绍到GRU模型。基本都围绕着seq2seq任务的这三种模型，也是本次实验我们所要做的核心工作。

Seq2Seq（强调目的）不特指具体方法，满足「输入序列、输出序列」的目的，都可以统称为 Seq2Seq 模型。

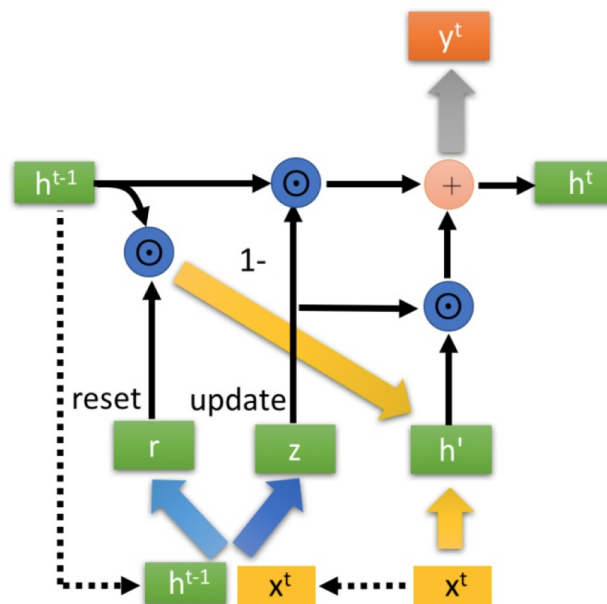


在序列到序列模型 (Seq2Seq) 中编码器Encoder: 将原句子转换为表征向量 h_{encoder} ; 解码器Decoder: 基于编码器得到的表征向量生成目标句子, 之前一般通过RNN进行编码和解码。因为Seq2Seq训练过程 (encoder) 会把所有信息映射到一个低维空间向量, 然后用唯一——一个低维空间向量做解码, 那么这个低维空间表征向量对原句子依赖性极大, 就很容易过拟合。

GRU

早期, RNN(循环神经网络)通过循环连接隐藏层的方式处理序列数据, 但难以捕捉长距离依赖关系, 容易出现梯度消失或梯度爆炸问题。LSTM(长短期记忆神经网络)是一种特殊类型的RNN, LSTM引入了三个门控单元, 这些门控机制通过学习的方式来控制信息的流动, 使网络能够选择性地记住或忽略输入数据中的信息, 能够更好地捕捉和处理长距离依赖关系, 使其在处理长序列任务时表现更优越, 避免了梯度问题。GRU也是一种改进的RNN模型。只有两个门 (重置门, 更新门), 整体的架构比LSTM简单, 减少了参数的数目。GRU 的hidden state依旧没有参与矩阵乘法运算 (与外界相对隔离), 避免了一定程度的梯度消失和梯度爆炸。工业界和学术界通常更多使用 GRU, 因为训练速度较快。

GRU 背后的原理与 LSTM 非常相似, 即用门控机制控制输入、记忆等信息而在当前时间步做出预测。GRU 有两个门, 即一个重置门 (reset gate) 和一个更新门 (update gate)。从直观上来说, 重置门决定了如何将新的输入信息与前面的记忆相结合, 更新门定义了前面记忆保存到当前时间步的量。如果我们将重置门设置为 1, 更新门设置为 0, 那么我们将再次获得标准 RNN 模型。

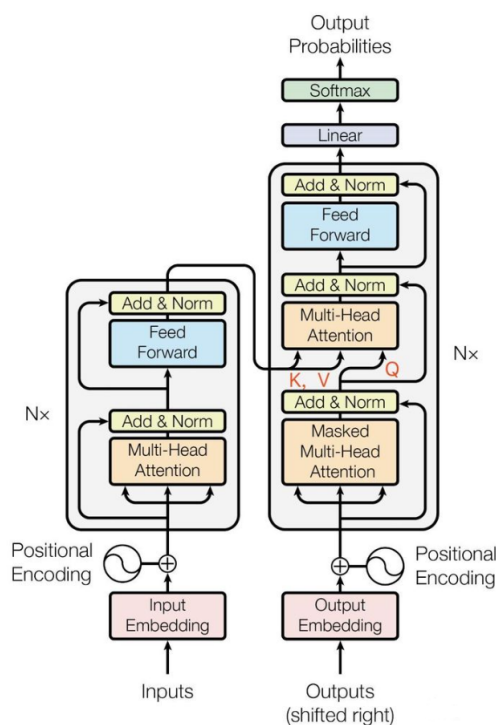


本次实验中, 我的GRU利用Encoder最后一个时间步来初始化Decoder的状态, 这是比较简单易操作的。

Transformer架构

Attention注意力机制给每个词语赋予了不同的权重表示重点，解决了Seq2seq的一些问题，不依赖于整个句子的表征，能够刻画生成的词和之前的哪个词关联度最大，并根据关联度给予更高权重，加权相加生成context vector。将context vector和隐状态拼接用于预测。

2017年，在Vaswani的论文"Attention Is All You Need"中，提出了自注意力机制(self attention)，它不仅能用在Seq2seq场景，而且是通用的深度学习技术，也是Transformer架构的核心。**Transformer**是近几年来流行的架构，不使用循环连接，引入了自注意力机制，能够在整个序列中同时关注不同位置的信息，能有效捕捉输入序列的全局依赖关系，也能有效处理长距离依赖，更能够并行处理输入序列加速训练。Transformer由编码器和解码器(Encoder-Decoder)组成，每个都包含多头注意力机制和前馈神经网络。多头注意力允许模型同时关注不同位置的信息，增强了对上下文的理解；位置编码则使模型能处理序列信息。Transformer的训练使用自监督学习，通过遮蔽语言模型和预测下一个词来学习丰富的表示。这种结构灵活，可用于各种任务，如机器翻译、文本生成和语言理解。Transformer成功推动了深度学习在自然语言处理领域的进展，成为现代 NLP 的基石。Transformer架构的大致示意图如下所示，也是Encoder-Decoder结构：



1. encoder

1. 首先 multi-head self-attention layer 多头注意力机制
2. 通过 Feed forward 增加非线性信息
3. Add和Norm，后续处理（像Residual connection、layer normalization等）
 - NLP很少做batch norm（不稳定），而是做layer norm
4. $N \times$ 表示乘做N次

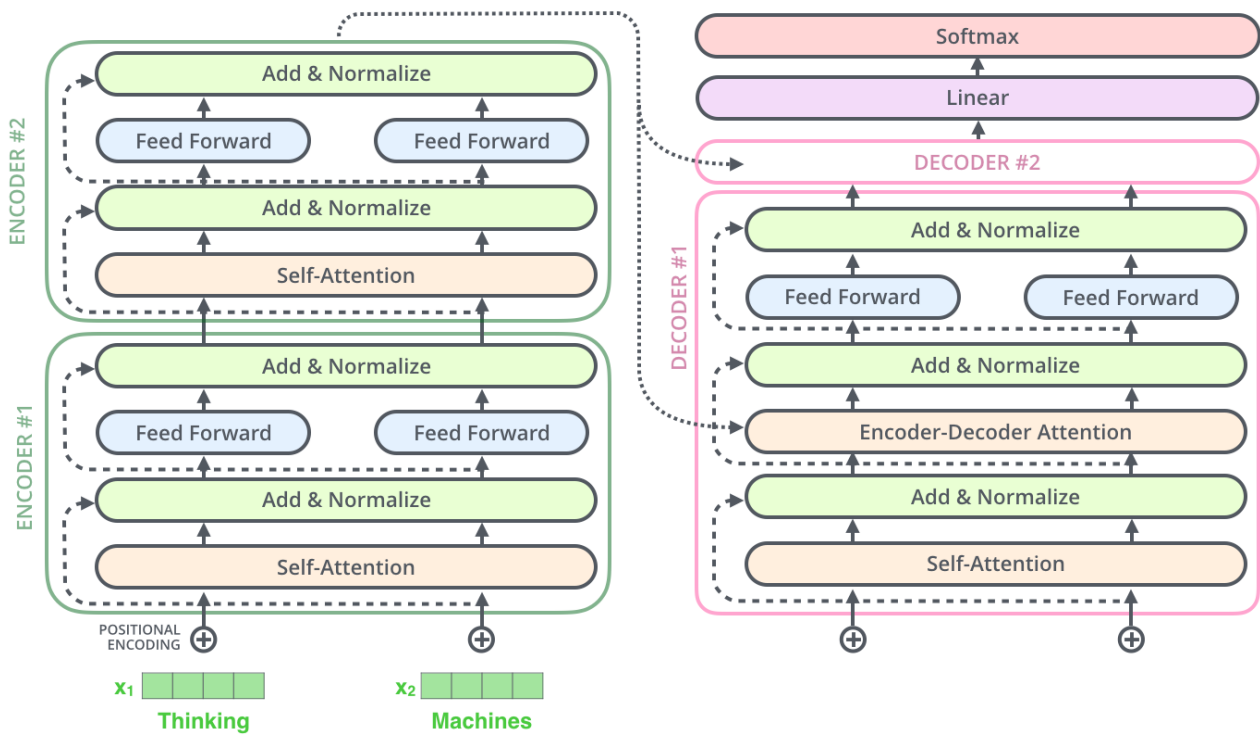
2. decoder

1. **Masked** multi-head self-attention layer 由于依次输出，所以mask尚未生成的单词。
2. **cross**-attention layer 在Decoder和Encoder之间两两进行Attention（encoder: keys/values, decoder: queries）
3. 通过 Feed forward 增加非线性信息

虽然大框架是类似的，但Transformer模型在细节上存在一些差异。比如说，T5完整地使用了Encoder-Decoder结构，GPT却只使用了Decoder结构。对于不同的模型，我们从原理入手，应该深入理解它们之间的异同之处。

T5

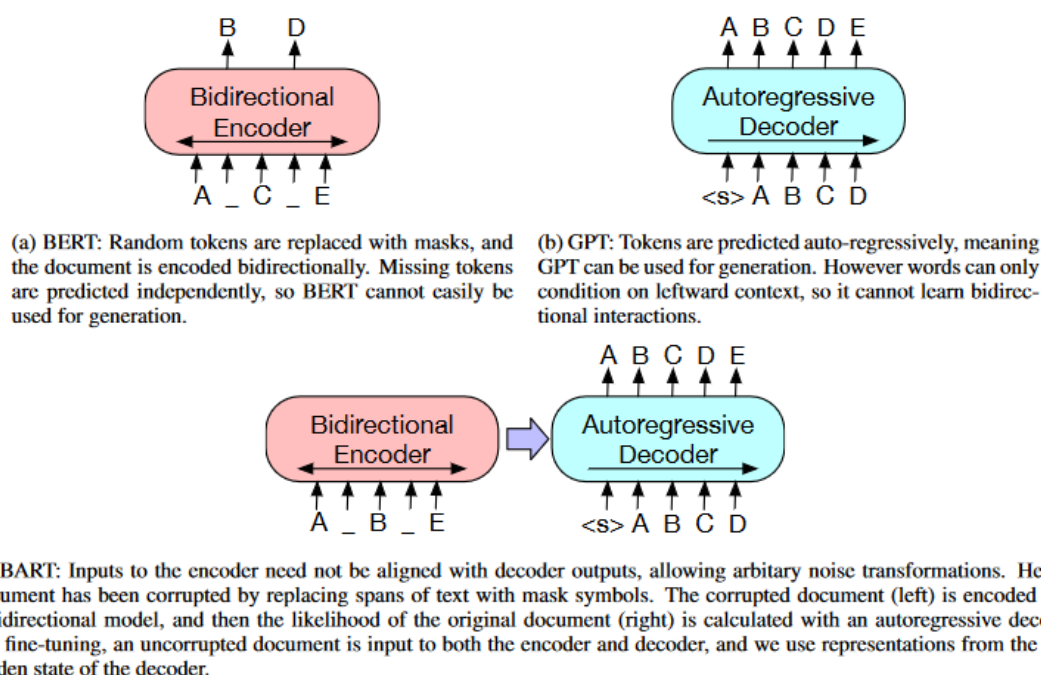
T5 (Text-to-Text Transfer Transformer) 主要思想是将所有的自然语言处理任务都转化为文本生成任务，使用了Transformer 的编码器-解码器结构。具体来说，T5的输入文本是任务的描述，输出文本是任务的解决方案。这使得T5非常灵活，可以用于各种自然语言处理任务，包括问答、摘要、翻译等。



T5在大规模文本上进行了预训练，使用了类似于BERT的Masked Language Model (MLM) 任务，但不同于BERT的是，T5的目标是将整个输入文本转化为整个输出文本。在预训练之后，T5可以通过微调来适应特定任务，微调的方式是在预训练的基础上添加任务特定的头部。

BART

BART (Bidirectional and Auto-Regressive Transformers) 结合了编码器-解码器框架和自回归 (auto-regressive) 训练方法。



Bart吸取了Bert的bidirectional encoder和GPT的left-to-decoder各自的特点，建立在Seq2Seq模型之上，以Transformer为基础。这些特点，不仅使得它比BERT适合文本生成，而且相比GPT多了获取双向上下文语境信息的环节。BART 虽然可以理解成是一个BERT +GPT 的结构，但是相比于 BERT 单一的噪声类型（单一地使用 [MASK] token 替换这一种噪声），BART在encoder尝试了多种噪声，意图破坏掉这些有关序列结构的信息，防止模型去依赖这样的信息。

BART吸收了BERT 双向自编码器和 GPT 自回归解码器的功能，可以应用在多个下游任务中。本次实验我们主要是在做文本摘要，这属于其中一个下游任务——Sequence Generation（文本生成）

三、实验内容

数据预处理

数据集被脱敏了，数据集中的词都被转化为了数字，这些数字无法再重新转化回单词进行语义的学习。脱敏后的数据可以认为是一种全新的语言，我们可以通过模型的预训练来让模型熟悉这个数据。

我自定义了一个pytorch数据集MyDataset，用来处理原始数据的description和diagnosis，模仿的是HuggingFace官方的写法，最主要的是 `__getitem__` 方法，这个方法用于获取数据集中索引idx 对应的样本。`__getitem__` 方法使用分词器对描述性文本和诊断文本进行编码，设置好对应的参数——比如说`add_special_tokens`（告诉分词器要加入特殊标记，比如说一些预定义的保留字符）`max_length`（最大长度）`return_tensors`（设置为'pt' 表示返回PyTorch 张量格式。）`padding='max_length'`（将输入序列填充到指定的最大长度,这样保证了各个序列在填充后长度的一致）

在成功调用了两种模型之后，我又想要体会一下以循环神经网络为核心的GRU架构，但是网上并没有像tokenizer那样打包得很好的方法，所以我不得不从零开始，把理论课讲到的一些关键步骤——比如文本分词、填充、转化为张量形式乃至构建词表——手动实现。我又自定义了一个pytorch数据集MyDataset1，这个结构其实和MyDataset很相似，只不过加入了我对数据的处理过程。

最后，为了方便之后调用数据，使之为模型可接受的输入格式，我紧凑化了数据，在把输入给展平之后，将需要的数据打包成一个三键的字典。

这是直接使用tokenizer的数据处理流程：

```

def __getitem__(self, idx):
    description_encoding = self.tokenizer(
        self.description[idx],
        add_special_tokens=True,
        max_length=self.max_len,
        padding='max_length',
        truncation=True,
        return_tensors='pt'
    )
    diagnosis_encoding = self.tokenizer(
        self.diagnosis[idx],
        add_special_tokens=True,
        max_length=self.max_len,
        padding='max_length',
        truncation=True,
        return_tensors='pt'
    )
    # 紧凑化
    return {
        'input_ids': description_encoding['input_ids'].flatten(),
        'attention_mask': description_encoding['attention_mask'].flatten(),
        'labels': diagnosis_encoding['input_ids'].flatten().tolist()
    }

```

这是我手动实现的数据处理流程：

```

def __getitem__(self, idx):
    #先处理description
    description_list = [int(num) for num in self.description[idx].split()]
    description_len = len(description_list)
    description = description_list[:self.max_len]
    description = torch.tensor(description)
    description = torch.cat((description, torch.tensor([pad_token] * (self.max_len -
len(description))), dtype=torch.long)))
    #再处理diagnosis
    diagnosis_list = [int(num) for num in self.diagnosis[idx].split()]
    diagnosis_len = len(diagnosis_list)
    diagnosis = diagnosis_list[:self.max_len]
    diagnosis = torch.tensor(diagnosis)
    diagnosis = torch.cat((diagnosis, torch.tensor([pad_token] * (self.max_len -
len(diagnosis))), dtype=torch.long)))
    # 终止符
    description[description_len] = end_token
    diagnosis[diagnosis_len] = end_token
    return description, diagnosis, description_len, diagnosis_len

```

注意这里不要忘记了在文本的末尾添加终止符

正式读取train.csv和test.csv文件之后，我利用keras的train_test_split方法把训练集和验证集按4：1的比例进行了划分。注意这里我train_test_split的shuffle选项开启了，数据是打乱之后才被划分的。

通过观察，训练集和验证集的描述最大长度为148：


```
train_data = pd.read_csv('train.csv')
x = train_data['description'].values
MAX_LEN = max([len(i.split(' ')) for i in x])
print(MAX_LEN)
```

以此作为基准去输入到MyDataset里面交由tokenizer分词器进行对应的工作即可。预处理之后数据存储在train_dataset,val_dataset,test_dataset中。

为了方便GRU的训练，毕竟是自己手写的，我还把MyDataset1里面的train_dataset,val_dataset,test_dataset用torch的DataLoader方法按batch_size拆分开来装入train_loader, val_loader, test_loader

模型训练

在实现前两个模型时，根据Huggingface官方的教程，我们只需要完善各个组件然后调用即可。

我本来是想手写训练过程呢，后来自己尝试了挺久之后发现问题实在是太多了，最后只好放弃了。然后我使用的是transformers库的Seq2SeqTrainer作为训练器。

```
trainer = Seq2SeqTrainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=test_dataset,
    tokenizer=tokenizer,
    compute_metrics=compute_metrics,
    data_collator=DataCollatorForSeq2Seq(tokenizer=tokenizer)
)
```

其中，training_args是训练相关的参数，compute_metrics是计算指标的方法，用于计算模型在给定数据集上的性能指标。以T5为例，我的参数如下所示：

```
training_args = Seq2SeqTrainingArguments(
    output_dir="./T5summary",
    per_device_train_batch_size=args.batch_size,
    per_device_eval_batch_size=args.batch_size*4,
    gradient_accumulation_steps=4,
    logging_steps=8,
    evaluation_strategy="epoch",
    save_strategy="epoch",
    learning_rate=1e-4,
    num_train_epochs=args.epochs,
    predict_with_generate=True
)
```

predict_with_generate指定是否使用 generate 来计算生成指标（ROUGE，BLEU），这个我差点就忘了开，导致之后什么生成的指标也没有。然后其他的参数其实就是从命令行中输入的参数。logging_steps是记录log的步数，我取值为8。

我们传统的训练过程时使用的是下面几句话：

```
optimizer.zero_grad()
loss.backward()
optimizer.step()
```

但是有时候模型显存占用量太大，为了节省显存，可以采用梯度累积的方法间接增大batchsize，gradient_accumulation_steps调整的就是这件事：

```
loss += loss / gradient_accumulation_steps
loss.backward()

if step % gradient_accumulation_steps == 0:
    optimizer.step()
    optimizer.zero_grad()
    step += 1
```

梯度累积到一定的次数(gradient_accumulation_steps)之后，optimizer.step()根据累计的梯度更新网络参数，然后zero_grad()清空之前的梯度，为下一次梯度累加做准备。

假设原来的batch size=10,数据总量为1000，那么一共需要100 train steps，同时一共进行100次梯度更新。若是显存不够，我们需要减小batch size，我们设置gradient_accumulation_steps=2，那么我们新的batch size=10/2=5，我们需要运行两次，才能在内存中放入10条数据，梯度更新的次数不变为100次，那么我们的train steps=200

这个trick我在使用时，考虑到自己租的云主机已经有32GB的显存了，还是非常可观的，所以就只把gradient_accumulation_steps取为4了。

因为之后的选择，我这里只在训练过程中统计BLEU-4和ROUGE2的值。这两个指标我为什么选择以及原理在后文。

```
results = []
rouge1 = Rouge()
def compute_metrics(pred_labels):
    predictions, labels = pred_labels
    decode_preds = tokenizer.batch_decode(predictions, skip_special_tokens=True)

    labels = np.where(labels != -100, labels, tokenizer.pad_token_id)
    decode_labels = tokenizer.batch_decode(labels, skip_special_tokens=True)

    decode_preds = [" ".join(p) for p in decode_preds]
    decode_labels = [" ".join(l) for l in decode_labels]
    scores = rouge1.get_scores(decode_preds, decode_labels, avg=True)

    rouge_scores = scores["rouge-2"]["r"]
    bleu_scores = np.mean([sentence_bleu([l.split()], p.split(),
smoothing_function=SmoothingFunction().method4) for l, p in zip(decode_labels,
decode_preds)])
    result = {
        "rouge2r": rouge_scores,
        "bleu4": bleu_scores
    }
    results.append(result)
return result
```


首先，从 `pred_labels` 中解包得到模型的预测值 `predictions` 和实际标签 `labels`。`decode_preds`先经过 `tokenizer`将模型的预测值解码成文本（这里一定不能忘记了跳过特殊标记!!!），再转换为字符串列表。

然后，对于实际标签中的-100（-100是pytorch中的特殊值）位置要被填充为`tokenizer.pad_token_id`（用于填充的`token_id`）填充之后进行对实际标签进行解码，再转换为字符串列表。注意这个字符串列表不能直接比较，看过 `train.csv`或`test.csv`的我们都知道里面的数据要以空格作为分隔，所以这里还要手动加上空格。

计算rouge分数时，只需要使用rouge库的Rouge类进行对应的操作即可，这里要记得取平均分数（`avg=True`）
BLEU-4分数的计算要使用`SmoothingFunction().method4`

最后，`trainer.train()` 训练即可

对于GRU的训练，类似于上次实验训练神经网络，我写了一个训练的方法。考虑到类RNN架构的预测能力较弱，给出的结果不是特别好，并且某一个状态时的较坏预测会对之后很多的状态都连带着产生影响。所以我在实际实现过程中，并没有傻傻地用上一个状态的输出作为下一个状态的输入。这个实验其实是在有监督的学习，我们知道训练集上的数据标签是什么，所以我直接采用上一个状态的真实标签而不是预测标签来作为下一个状态的输入。

虽然我这个训练策略会加剧过拟合问题，但是考虑到医学影响诊断报告的规律性，所以整体来说训练集上适应得很好的模型应该在验证集和测试集上取得还不错的结果。

实验结果

要想在测试集上看看结果，只需要用`evaluate`库的函数即可：

```
# 在测试集上看看结果
test_result = trainer.evaluate(test_dataset)
print(test_result)
```

当然这也只能看到在测试集上的情况，我们要想看到训练过程中的情况，其实只用查看`trainer.state.log_history`即可。导出数据之后我三种模型损失函数曲线如下所示（蓝线是在训练集上的损失函数值，红点是在验证集上的损失函数值。因为这个训练过程是每一个epoch才到验证集上跑一下，所以epoch之间的验证集上loss值没有测量）：

我这次实验选择的指标是BLEU-4和ROUGE 2。

BLEU（Bilingual Evaluation Understudy，双语评估替换）用于比较候选文本翻译与其他一个或多个参考翻译的评价分数，是一个主要用在机器翻译等文本生成任务上的指标。BLEU有许多变种，根据n-gram可以划分成多种评价指标，常见的指标有BLEU-1、BLEU-2、BLEU-3、BLEU-4四种，其中n-gram指的是连续的单词个数为n。具体的计算公式如下所示：

$$bleu_n = \frac{\sum_{c \in candidates} \sum_{n-gram \in c} Count_{clip}(n-gram)}{\sum_{c' \in candidates} \sum_{n-gram' \in c'} Count(n-gram')}$$

神经网络生成的句子是candidate，给定的标准译文是reference。在分子中，第一个求和符号统计的是所有的candidate，因为计算时可能有多个句子，第二个求和符号统计的是一条candidate中所有的n-gram，而 $Count_{clip}(n-gram)$ 表示某一个n-gram在reference中的个数，故分子的含义就是所以整个分子就是在给定的candidate中有多少个n-gram词语出现在reference中。类似于分子，分母的含义是获得所有的candidate中n-gram的个数。

BLEU这个指标计算速度快、与具体语言无关，但是它不考虑语言表达乃至语法上的准确性、测评精度会受常用词的干扰、没有考虑同义词或相似表达的情况。

Rouge(Recall-Oriented Understudy for Gisting Evaluation), 是评估自动文摘以及机器翻译的一组指标。它通过将自动生成的摘要或翻译与一组参考摘要（通常是人工生成的）进行比较计算，得出相应的分值，以衡量自动生成的摘要或翻译与参考摘要之间的“相似度”。

ROUGE-N的计算公式如下所示：

$$\text{ROUGE-N} = \frac{\sum_{S \in \{\text{Reference Summaries}\}} \sum_{gram_n \in S} \text{Count}_{match}(gram_n)}{\sum_{S \in \{\text{Reference Summaries}\}} \sum_{gram_n \in S} \text{Count}(gram_n)}$$

ROUGE-1只关注单个词的匹配，而ROUGE-L考虑整个子序列的匹配，因此它们更可能取得较高的分数。ROUGE-2考察的是2-gram的重叠，更注重对相邻单词对的匹配，但是这次的医学数据又经常出现跨越多个单词的专业术语，所以ROUGE-2分数一般较低。

ROUGE-L的计算公式如下所示：

$$R_{lcs} = \frac{LCS(X, Y)}{m}$$

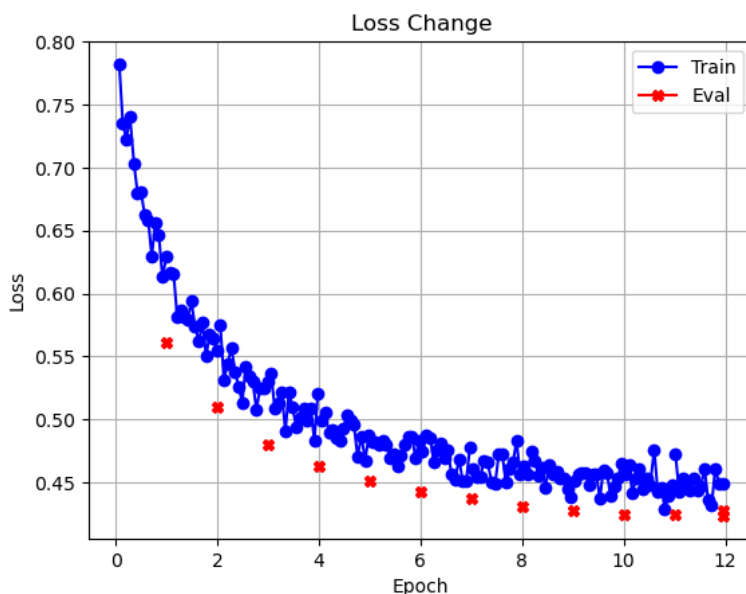
$$P_{lcs} = \frac{LCS(X, Y)}{n}$$

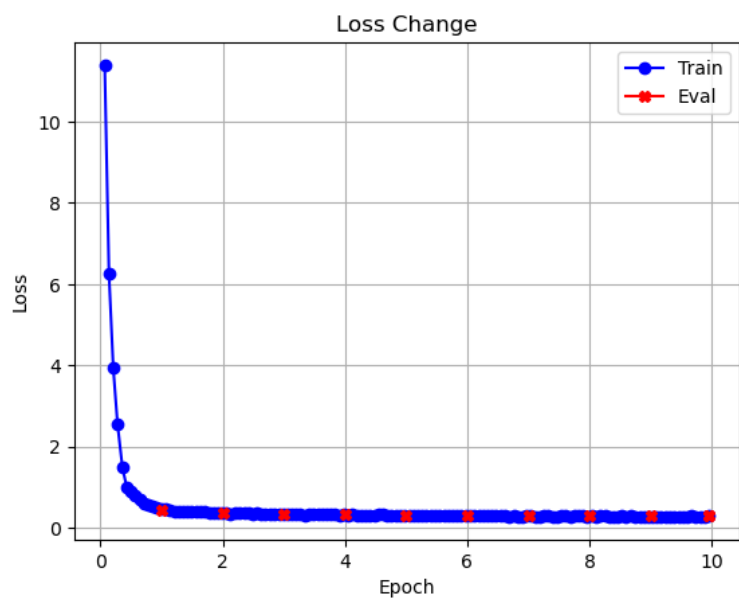
$$F_{lcs} = \frac{(1 + \beta^2) R_{lcs} P_{lcs}}{R_{lcs} + \beta^2 P_{lcs}}$$

具体的计算过程我就不赘述了。本次实验中我选择的主要指标是ROUGE2-R（召回率）和BLEU-4

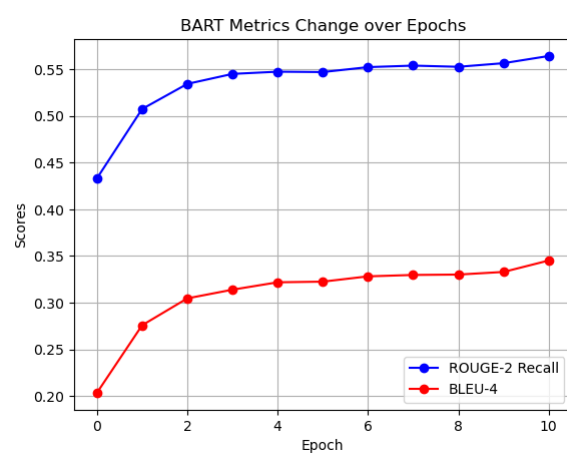
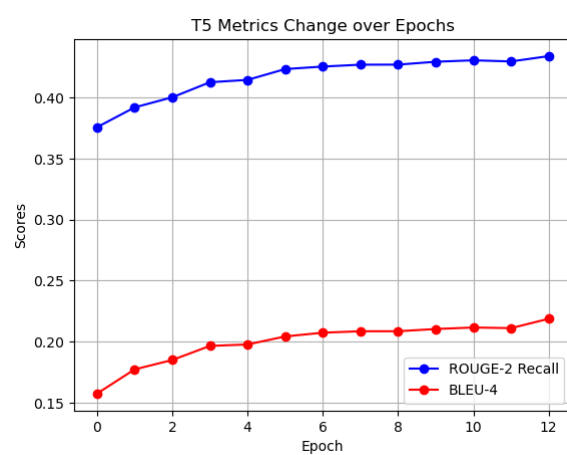
由于自己让模型并发地在GPU上跑存在一些问题，所以GRU在测试集上生成的速度较慢，因此我的GRU并没有每个epoch都去检验一下这两种指标。这两种指标根据定义来看，显然是数值越高反映模型的效果更好。

T5和BART这两种模型在训练过程中损失函数的值：





T5和BART这两种模型在训练过程中于验证集上的两种指标值：



在测试集上三者的结果：

	BLEU-4	ROUGE2-R
T5	0.21874	0.43375
BART	0.34512	0.56425
GRU	0.17432	0.21544

可以看到GRU的表现在三个模型中如理论所预期，即便训练的轮数最多（epochs=20），但也是最差的。

碰到的问题总结

1. Huggingface在国内被墙了，模型无法下载

下载模型时碰到了连接错误。

```
...
~/anaconda3/lib/python3.9/site-packages/transformers/utils/hub.py in get_from_cache(url, cache_dir, force_download, proxies, etag_timeout, resume_download, user_agent, use_auth_token, local_files_only)
552     )
553     else:
--> 554         raise ValueError(
555             "Connection error, and we cannot find the requested files in the cached path."
556             " Please try again or make sure your Internet connection is on."
557         )

ValueError: Connection error, and we cannot find the requested files in the cached path. Please try again or make sure your Internet connection is on.
```

在本地，我可以通过连接VPN解决这个问题。但是在云主机上，因为我对Ubuntu的相关配置实在是不熟悉，搞了很久这个问题也没有解决。所以我采用了比较笨的办法——在本地先把模型下载好了，然后再通过scp上传到服务器上，加载模型时加载本地文件即可。

2. OutOfMemoryError

```
OutOfMemoryError: CUDA out of memory. Tried to allocate 178.00 MiB. GPU 0 has a total capacity of 31.74 GiB of which 105.12 MiB is free. Including non-PyTorch memory, this process has 29.56 GiB memory in use. Process 1532 has 2.07 GiB memory in use. Of the allocated memory 28.92 GiB is allocated by PyTorch, and 288.99 MiB is reserved by PyTorch but unallocated. If reserved but unallocated memory is large try setting max_split_size_mb to avoid fragmentation. See documentation for Memory Management and PYTORCH_CUDA_ALLOC_CONF
```

这个问题主要是因为自己batch_size设置得太大了，即便有32G的显存，也分配不下了，调小一些batch_size即可。

3. ValueError: Hypothesis is empty.

```
.....
File ~/anaconda3/lib/python3.11/site-packages/rouge/rouge_score.py:253, in rouge_n(evaluated_sentences, reference_sentences, n, raw_results, exclusive)
235 """
236 Computes ROUGE-N of two text collections of sentences.
237 Source: http://research.microsoft.com/en-us/um/people/cyl/download/
(...)
250 ValueError: raises exception if a param has len <= 0
251 """
252 if len(evaluated_sentences) <= 0:
--> 253     raise ValueError("Hypothesis is empty.")
254 if len(reference_sentences) <= 0:
255     raise ValueError("Reference is empty.")

ValueError: Hypothesis is empty.
```

为什么我调用Rouge来计算时出现了这样的错误呢？看起来好像是去计算的语句之长度竟然非正了，我觉得可能是不小心调整错了导致显存溢出。经过一段时间的检查我发现自己training_args中设置的有些问题。

per_device_train_batch_size指定用于训练的每个 GPU/CPU 的 batch size；per_device_eval_batch_size指定用于评估的每个GPU/CPU的batch size 。然后我把per_device_eval_batch_size设置为了per_device_train_batch_size的四倍，而不是最初的相等，再去训练问题就解决了。

但是在使用T5模型时，我发现时常会有预测值pred为空值，这导致了Hypothesis is empty这个问题的发生。我在网上苦苦查询资料之后发现并没有很好地解决这个问题，于是根据官方的示例针对T5重新写了一段代码，放在了T5.py文件中。

4. 云主机配置环境的问题

我最开始租的云主机驱动版本是nvidia460.106，cuda版本11.2。我本地的云主机驱动版本是nvidia537.58，cuda版本12.2。在本地我能运行的代码，到了云主机上之后就出现各种很离谱的报错。后来我发现是torch的问题之后想要升级，结果因为云主机驱动程序我无法安装而不能设置。甚至我想通过conda/pip来upgrade时候给我从cuda版的包升级成了cpu版的(版本号最开始+cu结尾，更新之后却成了+cpu)

后来，焦灼了几个小时无果之后，我找到了最新版本的云主机镜像，驱动版本535.104，cuda版本12.2，再去安装和我本地机器对应的包之后再运行，终于解决了问题。

5. ValueError: Expected input batch_size (288) to match target batch_size (8).

最开始我没有按照比较简单的写法，反而是自己去写训练过程，这是自己就碰到了这样的问题。要解决其实也很简单，就是修数据集的格式代码：

```
train_transform = transforms.Compose([
    transforms.Resize((224,224)),
    transforms.RandomVerticalFlip(),
    transforms.ToTensor(),
    normalize
])
test_transform = transforms.Compose([
    transforms.Resize((224,224)),
    transforms.ToTensor(),
    normalize
])
```

然而自己后来干脆就使用了更为简便的模型微调方法，凭借官方的Seq2SeqTrainer，这个问题的解决也没有那么有用了。

6. Huggingface官方示例代码采用的DatasetDict数据结构之问题

在Huggingface官方网页，他们采用的是DatasetDict将训练集，验证集和测试集都整合在了一起然后进行训练。但是之后的实验中我发现这样做并不是很方便，比如说这个DatasetDict不支持keras的train_test_split方法划分数据集，而且很多地方经常报ValueError的错误。

后来，我在用pandas导入数据之后，自己写了个类来存数据（也是效仿着官方的示例，只不过是另一套）问题就解决了

7. 在验证集上测试时报错UserWarning: Input length of input_ids is 148, but max_length is set to 20.

这个主要是因为tokenizer默认的最大长度仅为20，这个我只需要修改它的max_new_tokens选项即可：

```
tokenizer = AutoTokenizer.from_pretrained("./tokenizer_xlnet",max_new_tokens=148)
```

但是改了之后还是报错，只不过不影响正常运行罢了。

进一步的分析

根据实验结果我们可以看到，基于Transformer架构的模型比基于RNN架构的模型确实在效果上有很大的提升。由于我采用的两种Transformer架构的模型都是预训练的，其中具体的机制难以去调整，相信根据实际的情景，我们手动来设计对应的模型，取得结果能更进一步。

而且，BART和T5相对于GRU来说都是超级大的模型，在如今这个大模型时代，虽然计算的成本更大了，但是带来了模型更加优秀的表现。

四、实验总结

在本次实验中，我以文本摘要任务为起点，通过T5,BART和GRU这三种模型，在医疗诊断数据集上进行微调并取得了还行的结果，我对几个预训练模型的原理、训练和验证过程有了更深刻的认识。结合理论课上的深入学习，我在明确传统的循环神经网络架构局限性的基础上，对最前沿的Transformer架构的了解和掌握更进一步了，这对之后我跟上人工智能的时代潮流打下了非常坚实的基础。